INTRODUCTION 정합성과 유지보수성을 우선하는 Back-End 개발자

Back-End Developer

서비스의 필요성과 구현 가능성을 함께 따지며, 데이터 정합성·유지보수성·안정성을 우선해 설계하는 백엔드 개발자입니다.

서버 설계 | 인프라 구축 | 금융 데이터 | 실시간 시스템

Certification / Awards

- **신한 해커톤 대상** — Shinhan Bank Hackathon with SSAFY (2025)
- **SQLD** — SQL Developer (2024)

Education

- **SSAFY** — 삼성 청년 SW 아카데미 · Java · Algorithm · REST API · DB · AI 교육 이수

Skills

백엔드

Spring Boot	■■■■■	REST API 설계, 도메인 분리, Swagger 문서화와 배포 경험
JPA	■■■■□	연관관계 매핑, 트랜잭션 경계 설정, 정합성 중심 모델링
Security	■■■□□	JWT 인증·인가, 필터 체인 설정, 권한 예외 처리 경험

데이터베이스

MySQL	■■■■□	정규화, 인덱스, JPA 기반 CRUD·조회 설계 경험
PostgreSQL	■■■■□	거래·예산·목표 데이터 모델링과 주간·월간 집계 처리
Redis	■■■□□	캐시와 HOT 데이터 분리로 빠른 조회 응답 보조

인프라 / DevOps

Docker	■■■■□	compose 기반 실행 환경 통일과 서비스 묶음 운영
Linux	■■■■□	Ubuntu 서버 배포, 로그 확인, 프로세스 운영 경험
Nginx	■■■■□	리버스 프록시와 정적 자산·API 라우팅 설정
CI/CD	■■■□□	GitHub Actions로 빌드·배포 자동화 구성
AWS S3	■■■□□	이미지·음성 업로드 저장소 분리와 경로 정책 설계

언어

Java	■■■■□	객체지향 설계 기반 유스케이스·서비스 로직 구현
JavaScript	■■■□□	포트폴리오·프론트 연동용 UI 스크립트 수정 및 배포 대응
Python	■■■□□	분석 API 연동, 데이터 전처리, AI 파이프라인 작업 경험

AI / 데이터

Vector Sea…	■■■□□	시맨틱 검색 구현
Embedding	■■■□□	벡터화, 검색 전처리
Whisper/TTS	■■□□□	음성 기반 서비스 연동

PROJECT 01 CAMPUNG!

CAMPUNG! ★ 해커톤 대상 백엔드 / 인프라

신한 해커톤 with SSAFY · 2025.08.13 ~ 2025.08.31 · 지도 기반 캠퍼스 커뮤니티

프로젝트 소개

- 텍스트 게시판 중심 커뮤니티의 한계 때문에 캠퍼스 현장 이슈를 즉시 공유하기 어려운 문제를 해결하고자 기획한 서비스
- 지도에서 바로 공유하고 게시판에 구조화해 남기는 흐름으로 현장성과 지속성을 함께 확보

해결 방식

- 게시글, 랜드마크, 캠퍼스 온도 API를 분리해 지도와 게시판이 같은 데이터를 재사용하도록 구성
- AI 감정 분석 결과를 온도 지표와 연결해 게시글 흐름을 한눈에 보여주는 서비스 지표 설계
- 파일 업로드 경로를 Content-Type 기준으로 분기해 이미지와 녹음 파일 운영 경로를 분리

담당 역할

- 게시글, 랜드마크, AI 감정 분석 관련 백엔드 API 설계 및 구현
- 게시글 온도 집계 로직 구현
- S3 기반 이미지·녹음 파일 업로드 구조 구성
- VMware Ubuntu Live Server 기반 배포 환경 구축 및 CI/CD 구성

사용 기술

Spring Boot | WebFlux | MariaDB | Redis | AWS S3 | Docker | Nginx | GitHub Actions | OpenAI

CAMPUNG 1

1 / 4

기술 선택 이유

- Spring Boot를 중심으로 게시글·랜드마크·알림 도메인을 나누고, 익숙한 MVC 구조 위에서 빠르게 기능을 확장
- WebFlux는 서버 전체 전환보다 GPT API 호출용 WebClient에 집중해 외부 AI 연동 지연을 분리
- MariaDB, Redis, S3를 영속 데이터·빠른 조회·파일 저장으로 역할 분리해 운영 구조를 단순화
- 개인 PC 기반 Ubuntu Live Server와 GitHub Actions로 짧은 해커톤 기간에도 반복 가능한 배포 환경 확보

성과 / 결과

- 게시판 텍스트를 읽기 전에도 캠퍼스 분위기를 지표로 먼저 파악할 수 있는 구조로 전환
- 이미지·녹음 파일 저장 경로를 유형별로 분리해 운영 중 파일 탐색과 장애 대응 범위를 좁힘
- 신한 해커톤 대상 수상으로 문제 정의와 구현 완성도를 외부 평가로 검증

트러블슈팅 CAMPUNG!

1. 캠퍼스 온도 집계 — AI 감정 분석 + 양방향 온도 보정

게시글 목록만으로는 캠퍼스 분위기를 직관적으로 전달하기 어려웠습니다.
AI 감정 분석 결과를 활동량과 연결해 0~100°C 범위의 캠퍼스 온도 지표를 구현하되,
시간대별 극단값 방지와 보호 계수를 적용해야 했습니다.

[양방향 온도 보정 로직]

```java
private double calculateBidirectionalAdjustment(
    double currentTemp, int postCount, int hour) {

  PostActivityLevel activityLevel =
      postActivityAnalyzer.analyzeCurrentActivity(post
  double baseRate = activityLevel.getAdjustmentRate();
  double protection = getTemperatureProtectionFactor(
      currentTemp, activityLevel.isIncreasing());

  TemperatureGuideline guideline =
      guidelineConfig.getGuideline(hour);
  double timeMult = activityLevel.isIncreasing()
      ? guideline.getIncreaseMultiplier()
      : guideline.getDecreaseMultiplier();

  return currentTemp * (1 + baseRate * protection * ti
}
```

[설계 포인트]

- 활동량 분석 → 상승/하강 판단
- 보호 계수로 극단값(0°/100°) 접근 시 변화폭 자동 감소
- 시간대별 가이드라인으로 낮/밤 보정 배율 분리

- **게시글 목록을 읽기 전에도 홈 화면에서 캠퍼스 분위기를 한 번에 파악할 수 있는 지표를 제공**
- **보호 계수와 시간대 보정으로 게시글이 적은 시간에도 지표가 급변하지 않도록 제어**

---

2. S3 파일 업로드 — Content-Type 기반 폴더 자동 분류

이미지·녹음 파일을 동일 경로에 저장하면 운영·검색이 복잡해지는 문제가 있었습니다.
Content-Type을 분석해 자동으로 폴더를 분류하고, 날짜 기반 경로로 관리 효율을 높였습니다.

[S3 업로드 + 폴더 자동 분류]

```java
public String uploadFile(MultipartFile file) {
  String contentType = file.getContentType();
  String folder = determineFolder(contentType);
  String key = folder + "/contents/"
    + LocalDate.now().format(
        DateTimeFormatter.ofPattern("yyyy/MM/dd"))
    + "/" + generateFileName(
        file.getOriginalFilename());

  PutObjectRequest req = PutObjectRequest.builder()
      .bucket(bucketName).key(key)
      .contentType(contentType).build();
  s3Client.putObject(req,
    RequestBody.fromInputStream(
      file.getInputStream(), file.getSize()));
  return baseUrl + key;
}
```

[저장 구조 분리 결과]

- 영속 데이터 → MariaDB
- 캐시 / 실시간 조회 → Redis
- 이미지·녹음 파일 → S3 (Content-Type별 폴더)

[폴더 라우팅]

- image/* → images/contents/yyyy/MM/dd/
- audio/* → audio/contents/yyyy/MM/dd/

- **업로드 직후 생성되는 S3 key만으로 이미지와 녹음 파일 위치를 구분할 수 있어 운영 탐색 경로를 단순화**
- **DB·캐시·파일 저장소 책임을 나눠 장애 원인 파악 범위를 저장 계층별로 좁힘**

---

3. 프로젝트 회고

해커톤처럼 일정이 짧을수록 기능 구현보다 운영 가능한 구조를 먼저 잡아야 한다는 점을 배웠습니다.
반면 이번 프로젝트에서는 온도 지표와 파일 구조 개선 효과를 응답 시간이나 운영 시간 절감처럼 정량화하지 못한 아쉬움이 남았습니다.

- **배운 점: 사용자 경험을 높이는 기능도 결국 정합성, 저장소 분리, 극단값 제어 같은 운영 관점이 뒷받침돼야 한다는 점**
- **아쉬운 점: 구조 개선 효과를 설명할 로그·모니터링 지표까지는 연결하지 못함**
- **다음 개선: Redis 조회 지표, 업로드 실패율, 응답 시간 기록을 붙여 개선 효과를 정량 검증하고 싶음**

PROJECT 02 모아 (MOA)

모아 (MOA)

팀장 | 백엔드 / 인프라

SSAFY 특화 프로젝트 · 2026.02.18 ~ 2026.03.30 · 금융 Open API 기반 목표/예산 관리 서비스

프로젝트 소개

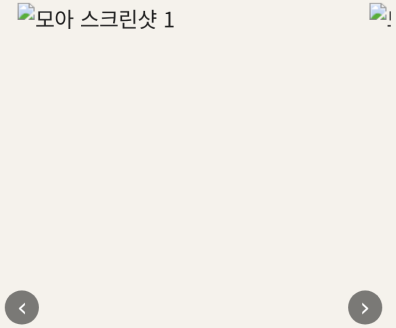
- 거래 데이터와 금융 Open API를 활용해 소비 분석, 목표 설정, 예산 관리를 하나의 흐름으로 연결한 금융 서비스
- 분석 결과를 보여주는 데서 끝나지 않고 목표 계획 수립과 주간 예산 제안까지 이어지는 실행형 사용자 흐름 설계

해결 방식

- 계좌 연동 이후 3개월 거래를 수집·분석하고, 목표 설정 결과를 주간 예산으로 다시 연결하는 흐름 설계
- 외부 금융 API 데이터와 로컬 DB를 reconcile해 분석·예산·목표가 같은 계좌 기준을 보도록 통합
- 월간 수입을 주차별 예산, 저축, 여유 자금으로 나누고 부족분은 우선순위가 낮은 목표부터 조정

담당 역할

- Account 파트 계좌 연동 및 관리 API 구현
- Budget 파트 예산 배분·할당 로직 구현
- 거래내역 동기화와 데이터 정합성 처리
- Docker 기반 인프라 환경 구성



1 / 4

사용 기술

Spring Boot | FastAPI | PostgreSQL | Docker | Ubuntu | SSAFY 금융 Open API

성과 / 결과

- 분석 결과 화면에서 끝나지 않고 목표 계획 수립, 주간 예산 설정, 홈 자유소비 관리까지 이어지는 실행 흐름을 구성
- 계좌 동기화 이후 분석·예산·목표 화면이 동일한 계좌 기준을 보도록 데이터 기준점을 통일
- 목표 충돌 시 부족분을 우선순위가 낮은 목표부터 조정해 금액 변화 이유를 설명 가능한 구조로 설계

기술 선택 이유

- Spring Boot를 메인 API 서버로 두어 계좌, 거래, 목표, 예산처럼 트랜잭션이 중요한 금융 도메인을 일관된 계층으로 관리
- PostgreSQL을 선택해 거래·예산·목표 관계와 주간·월간 집계를 안정적으로 다루고, Docker로 팀 전체 실행 환경을 통일
- FastAPI는 분석 응답처럼 계산 로직 변경이 잦은 영역을 분리할 때 유리해 금융 도메인 API와 분석 파이프라인의 책임을 나누는 구조에 적합

트러블슈팅 모아 (MOA)

1. 계좌 데이터 정합성 — 외부 API 동기화 (Reconciliation)

로컬 DB의 계좌 정보와 외부 금융 API 데이터 사이에 불일치가 발생하면
분석, 예산, 목표 화면 전체의 일관성이 깨지는 문제가 있었습니다.
외부 API 기준으로 비교·동기화하고, 삭제된 계좌는 비활성 처리하는 reconcile 구조를 구현했습니다.

[계좌 Reconciliation 로직]

```java
@Transactional
public List<Account> execute(Integer userId) {
  List<Account> linked =
    accountRepo.findAllLinkedByUserId(userId);
  User user = findUserUseCase.execute(userId);

  // □□ API□□ □□ □□ □□
  Map<String, AccountItem> external =
    apiClient.inquireAccounts(user.getUserKey())
      .accounts().stream()
      .collect(Collectors.toMap(
        AccountItem::accountNo, identity()));

  return linked.stream()
    .filter(acc -> reconcile(acc, external))
    .toList();
}

private boolean reconcile(Account acc,
    Map<String, AccountItem> external) {
  AccountItem ext = external.get(acc.getAccountNo());
  if (ext == null) {
    acc.markInactive(); return false;
  }
  acc.sync(ext); acc.markActive();
  return true;
}
```

[Reconciliation 포인트]

- 외부 API 기준 Map 구성 → O(1) 비교
- 존재하지 않는 계좌 → markInactive (삭제 대신 비활성)
- 메타데이터 변경 → sync로 일괄 갱신
- @Transactional로 부분 실패 방지

- **동기화 이후 홈·분석·목표가 동일한 계좌 목록을 기준으로 계산되도록 데이터 기준점을 통일**
- **삭제된 계좌를 즉시 제거하지 않고 비활성 처리해 외부 API 일시 오류에도 기존 이력을 보존**

---

2. 예산 배분 알고리즘 — 월간 수입 → 주간 배분 + 부족분 조정

단순 지출 분석만으로는 사용자가 행동으로 옮기기 어려웠습니다.
월간 수입을 주 단위로 분배하면서 고정비, 목표 저축, 여유 자금을 모두 반영하는 다단계 배분 로직을 구현했습니다.

[예산 배분 핵심 흐름]

```java
public AllocationResult allocateBudget(
    User user, LocalDate start,
    Long monthlyIncome,
    List<EssentialExpenseMonth> essentials,
    List<Plan> activePlans) {

  // 1. □□□
  long income = normalizeAmount(monthlyIncome);
  // 2. □ □□□ □□
  List<WeekFrame> weeks =
    buildWeekFrames(start, allocationEnd);
  // 3. □ □□ → □ □□
  Map<LocalDate,Long> incomeByWeek =
    allocateMonthlyIncome(start, end,
      income, weeks);
  // 4. □□□ □□
  List<EssentialAlloc> essentialAllocs =
    buildEssentialAllocations(...);
  // 5. □□□ □□
  applyShortageAdjustments(weeks,
    budgetWeeks, incomeByWeek,
    essentialByWeek, planAllocs);

  return new AllocationResult(...);
}
```

[배분 로직 구조]

- 월간 수입 → 주 단위 균등 분배
- 고정비(필수 지출) 우선 차감
- 목표 저축 계획 반영
- 부족분 발생 시 자동 조정(Shortage Adjustment)

[핵심 판단]

- 분석 결과 → 계획 수립 → 실행으로 이어지는 흐름 설계
- 조회 중심이 아닌 실행 중심 서비스로 전환

- **온보딩 분석 결과가 주간 예산, 목표 계획, 홈 자유소비 금액으로 이어지는 실행 흐름을 구성**
- **우선순위가 낮은 목표부터 부족분을 조정해 예산이 모자랄 때도 어떤 목표를 먼저 지킬지 설명 가능한 구조 확보**

---

3. 프로젝트 회고

금융 서비스는 계산 로직 자체보다도 같은 데이터를 기준으로 읽고 쓰게 만드는 구조가 더 중요하다는 점을 배웠습니다.
다만 예산 추천과 목표 계획이 실제 사용자 행동을 얼마나 바꿨는지까지 계측하지 못해 성과 설명은 아직 보강 여지가 있습니다.

- **배운 점: 금융 도메인에서는 화면별 기능보다 데이터 기준점과 동기화 순서를 먼저 맞추는 것이 안정성의 출발점이라는 점**
- **아쉬운 점: 예산 추천 효과를 목표 달성률이나 예산 초과 감소 같은 지표로 검증하지 못함**
- **다음 개선: 동기화 횟수, 목표 달성 진행률, 예산 초과 빈도를 수집해 추천 로직 성과를 정량화하고 싶음**

OTHER PROJECTS 기타 프로젝트 및 경험

HearBe

풀스택 보조 + AI

SSAFY 자율 프로젝트 · 2026.01.07 ~ 2026.02.09 · 시각장애인을 위한 음성 기반 웹 쇼핑 지원 서비스

HearBe 스크린샷 1

1 / 2

핵심 경험

- ASR(Whisper) → NLU → LLM → TTS 파이프라인을 연결해 음성 중심 사용자 흐름 설계
- WebSocket 기반 실시간 음성 세션 처리와 MCP 연동 경험 확보
- Jira API 기반 CLI 자동화 모듈을 만들어 협업 운영 효율 개선, 이후 모아 프로젝트에도 재사용

FastAPI | WebSocket | Whisper | GPT | TTS | MCP

TripTube

풀스택

SSAFY 관통 프로젝트 · 2025.11 ~ 2025.12.26 · 유튜브 여행 영상 자막을 AI로 구조화하고 시맨틱 검색을 제공하는 여행 계획 플랫폼

핵심 경험

- YouTube 자막 추출 → GPT 구조화 → 벡터 임베딩까지 이어지는 검색 파이프라인 구축
- 한국어 특화 임베딩 기반 시맨틱 검색 API를 설계·구현해 검색 품질 개선 경험 확보
- Spring Boot 백엔드와 AI 서버를 분리해 검색/추천형 기능을 서비스 API와 연결한 경험

Spring Boot | FastAPI | Vector DB | GPT | Upstage Embedding

Sequence

백엔드

대학 연합 프로젝트 · 2024.10 ~ 2025.06 · 대학 간 PM · 디자이너 · 개발자 교류 및 프로젝트 매칭 플랫폼

Sequence 스크린샷 1

1 / 2

핵심 경험

- 프로젝트 아카이빙과 상호 팀원 평가 기능을 중심으로 아카이브 파트 백엔드 개발
- QueryDSL 기반 동적 필터링·검색 쿼리로 복합 조회 조건 처리 경험 확보
- JPA 연관관계 설계와 soft-delete 기반 데이터 관리 패턴 적용

Spring Boot | JPA | QueryDSL | Security | Docker

블루로봇 인턴

프로젝트형 인턴

기업 인턴십 · 2025.06 ~ 2025.08 · 고정밀 균형 유지 하드웨어의 실시간 상태를 시각적으로 표현하는 2D 시각화 모듈 개발

핵심 경험

- Kotlin 기반 2D 시각화 모듈을 단독 개발하며 실시간 상태 표현 로직 구현
- 하드웨어 센서 데이터를 수신해 균형 상태를 그래픽으로 렌더링하는 인터페이스 개발 경험 확보

Kotlin | 2D Visualization